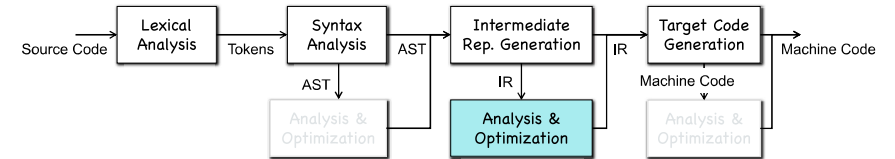


Chapter 12-1 Inter-procedural Analysis

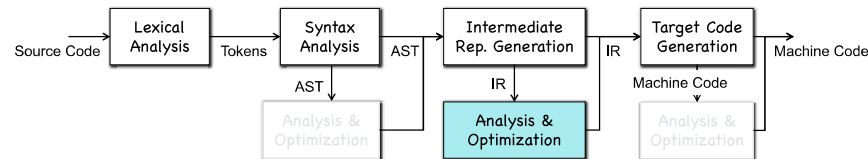
Data Flow Analysis (DFA)



- How do we define a DFA?
- What are flow-insensitive, flow-sensitive, and path-sensitive analysis?
- What are symbolic execution and concolic execution?

4

Inter-procedural DFA



- Why inter-procedural DFA?
 - virtual method invocation
 - detection of software defects
 - ...

```
public void foo(List l, int x) { l.add(x); }
```

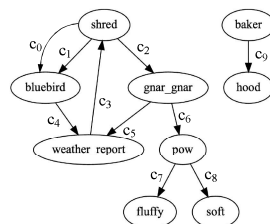
Bug #87203, which is detected by our work
36 functions over 11 compiling units



9

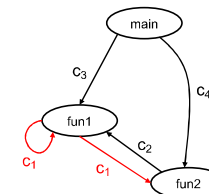
Call Graph (CG) A bit different from the definition in the dragon book.

- Nodes --- Functions
- Edges --- Call relations between functions
- Edge Labels --- Call sites



10

Call Chains



Call chains:
• c3-c1-...
• c4-c2-c1-...
• ...

```
int (*pf)(int);

int fun1(int x) {
    if (x < 10)
        return (*pf)(x+1);
    else
        return x;
}

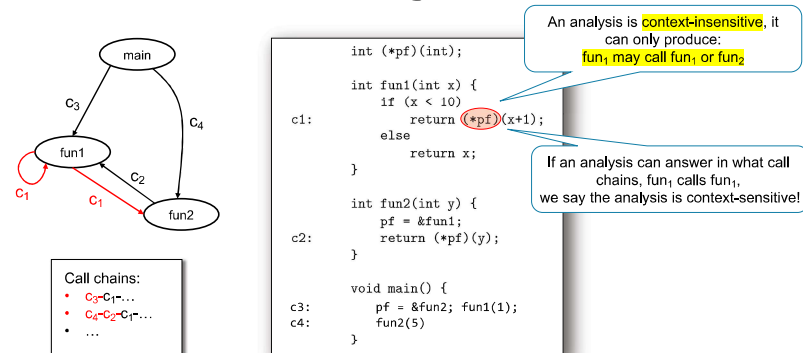
int fun2(int y) {
    pf = &fun1;
    return (*pf)(y);
}

void main() {
    c3: pf = &fun2; fun1(1);
    c4: fun2(5);
}
```

It may call different functions based on different calling contexts --- call chains (CG paths)!

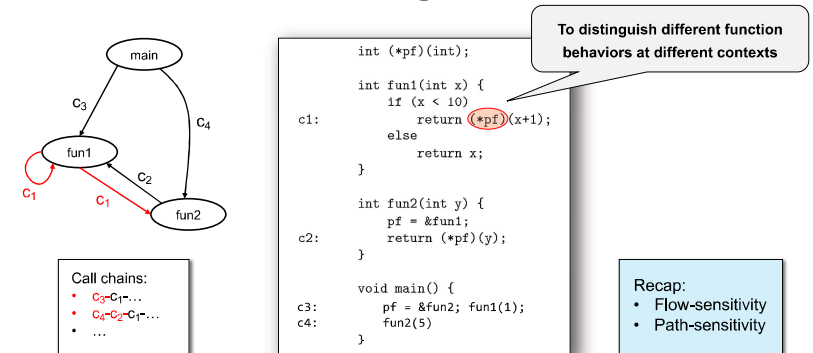
18

Context Sensitivity



21

Context Sensitivity

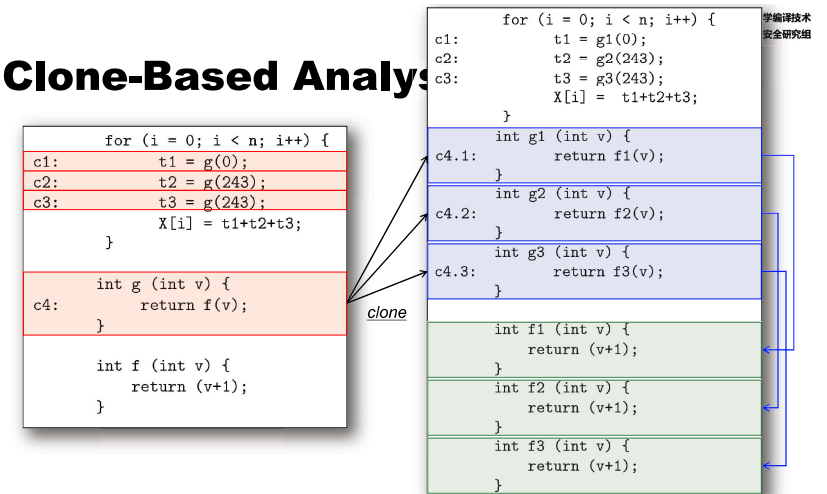


23

Context-Sensitive DFA

- **Clone-based** interprocedural DFA
- **Summary-based** interprocedural DFA

Clone-Based Analysis

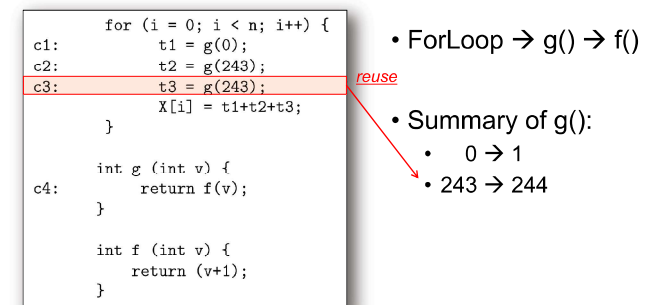


24

Context-Sensitive DFA

- **Clone-based** inter-procedural DFA
 - Let every function have only a single calling context
- **Summary-based** inter-procedural DFA
 - Top-down (TD) analysis: Analyze caller before callee
 - More specific to calling context
 - Bottom-up (BU) analysis: Analyze callee before caller
 - (i) Should assume all possible calling contexts; (ii) Easy to parallelize

TD Summary-Based Analysis



32

37

BU Summary-Based Analysis

```

for (i = 0; i < n; i++) {
c1:    t1 = g(0);
c2:    t2 = g(243);
c3:    t3 = g(243);
      X[i] = t1+t2+t3;
}

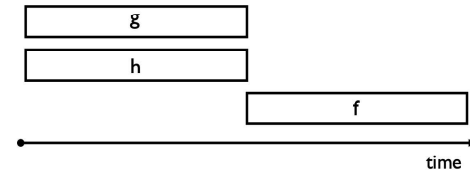
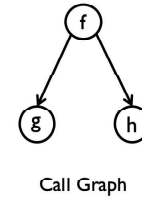
int g (int v) {
  return f(v);
}

int f (int v) {
  return (v+1);
}
    
```

- $f() \rightarrow g() \rightarrow \text{ForLoop}$
- *reuse*
- Summary of $f()$: $x \mapsto x + 1$
- Summary of $g()$: $x \mapsto x + 1$

41

Parallelizing BU Analysis



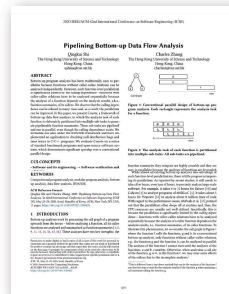
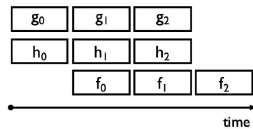
43

Pipelining BU Analysis

Pipelining Bottom-up Data Flow Analysis.

Qingkai Shi and Charles Zhang.

The ACM/IEEE International Conference on Software Engineering (ICSE'20).



44

PART II: DFA as Graph Reachability

Recap: Constant Propagation

- Each variable $v \rightarrow \text{UNDEF, Constant, NAC}$, denoted as $m(v)$

Transfer Function of $x = y$

- $m(y) = \text{Constant}$
• $\Rightarrow m(x) = \text{Constant}$
- $m(y) = \text{UNDEF}$
• $\Rightarrow m(x) = \text{UNDEF}$
- otherwise
• $\Rightarrow m(x) = \text{NAC}$

Transfer Function of $x = y + z$

- $m(y) = \text{Constant}, m(z) = \text{Constant}$
• $\Rightarrow m(x) = \text{Constant}$
- either of $m(y)$ and $m(z)$ is UNDEF
• $\Rightarrow m(x) = \text{UNDEF}$
- otherwise
• $\Rightarrow m(x) = \text{NAC}$

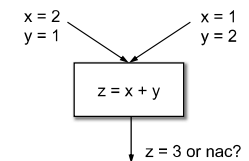
48

Recap: Constant Propagation

- Non-distributivity: Meet-Then-Transfer $\neq$ Transfer-Then-Meet

Merge Function: $m(x) \wedge m(x')$

- $\text{UNDEF} \wedge \text{Constant} = \text{Constant}$
- $\text{NAC} \wedge \text{Constant} = \text{NAC}$
- $\text{Constant}_1 \wedge \text{Constant}_1 = \text{Constant}_1$
- $\text{Constant}_1 \wedge \text{Constant}_2 = \text{NAC}$



56

Possibly Uninitialized Variables

- A statement uses a possibly uninitialized variable if there is a path to the statement without any definition of the variable.

57

Possibly Uninitialized Variables

- Each variable $v \rightarrow \text{UNDEF}, \text{Constant}, \text{NAC}, \text{DEF}$, denoted as $m(v)$

Merge Function: $m(x) \wedge m(x')$
• $\text{UNDEF} \wedge \text{Constant} = \text{Constant}$
• $\text{NAC} \wedge \text{Constant} = \text{NAC}$
• $\text{Constant}_1 \wedge \text{Constant}_2 = \text{Constant}_1$
• $\text{Constant}_1 \wedge \text{Constant}_2 = \text{NAC}$

Constant Propagation

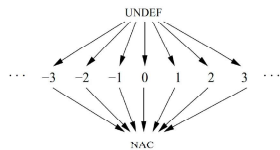
Merge Function: $m(x) \wedge m(x')$
• $\text{UNDEF} \wedge \text{DEF} = \text{UNDEF}$
• $\text{DEF} \wedge \text{DEF} = \text{DEF}$
• $\text{DEF} \wedge \text{DEF} = \text{DEF}$
• $\text{DEF} \wedge \text{DEF} = \text{DEF}$

Uninitialized Variables

61

Possibly Uninitialized Variables

- Finity: Possible dataflow values are finite



Constant Propagation



Uninitialized Variables

66

Possibly Uninitialized Variables

- Each variable $v \rightarrow \text{UNDEF}, \text{Constant}, \text{NAC}, \text{DEF}$, denoted as $m(v)$

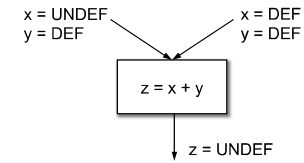
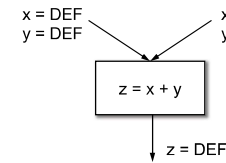
Transfer Function of $x = y$
• $m(y) = \text{DEF}$
• $\Rightarrow m(x) = \text{DEF}$
• $m(y) = \text{UNDEF}$
• $\Rightarrow m(x) = \text{UNDEF}$

Transfer Function of $x = y + z$
• $m(y) = \text{DEF}, m(z) = \text{DEF}$
• $\Rightarrow m(x) = \text{DEF}$
• either of $m(y)$ and $m(z)$ is UNDEF
• $\Rightarrow m(x) = \text{UNDEF}$

60

Possibly Uninitialized Variables

- Distributivity: $f(x \cup y) = f(x) \cup f(y)$



65

IFDS Problem

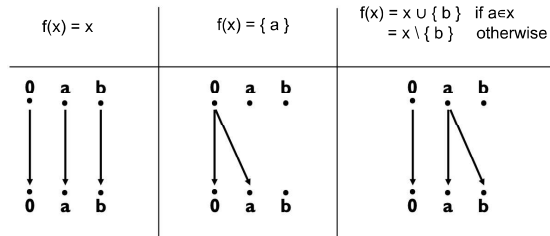
- Interprocedural Finite Distributive Subset Problem

Precise Inter-procedural Dataflow Analysis via Graph Reachability
Reps, Horwitz, Sagiv, POPL 1995



67

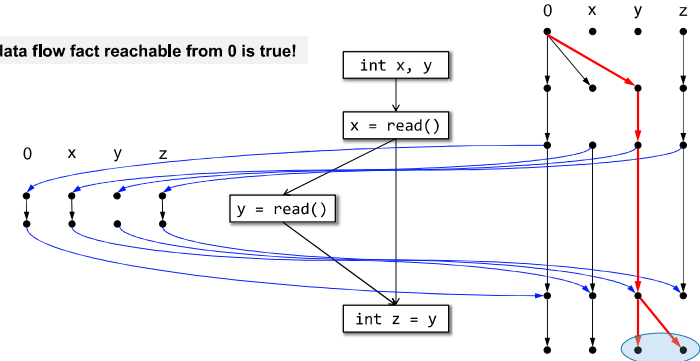
IFDS as Graph Reachability



68

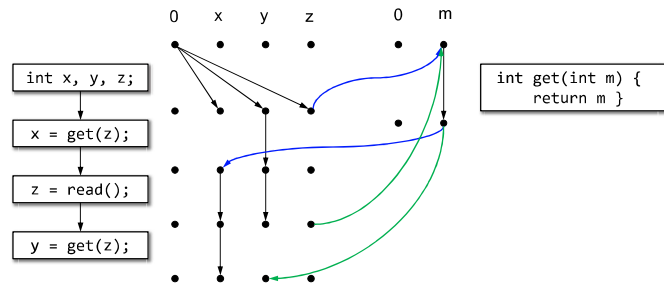
IFDS as Graph Reachability

Any data flow fact reachable from 0 is true!



85

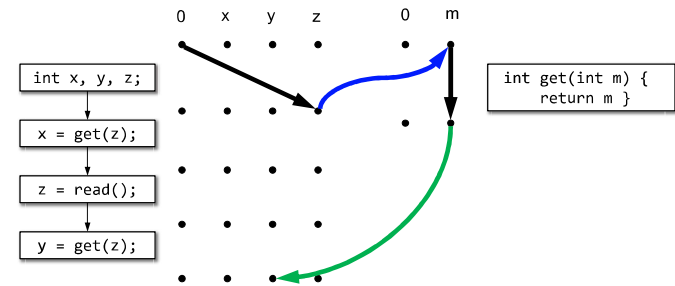
IFDS as Graph Reachability



PS: Edges between 0s are omitted

93

Context-Sensitivity



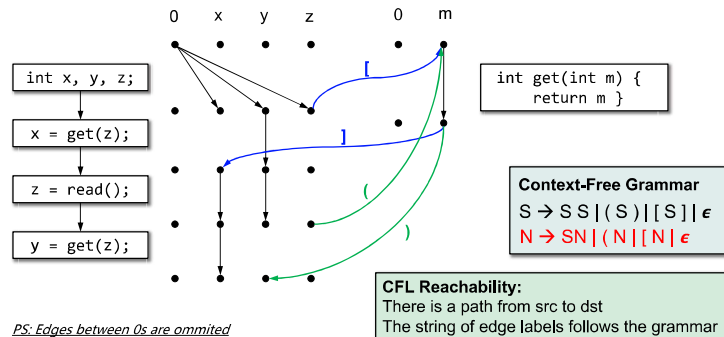
PS: Edges between 0s are omitted

y is uninitialized?

No! Why?

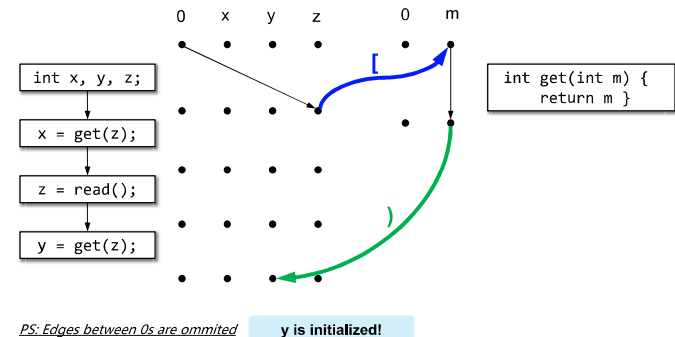
98

CFL Reachability



PS: Edges between 0s are omitted

CFL Reachability



PS: Edges between 0s are omitted

y is initialized!

100

Computing Transitive Closure

- **Time:** $O(|V|^3/\log|V|)$; **Space:** $O(|V|^2)$



Subcubic algorithms for recursive state machines
Swarat Chaudhuri, POPL, 2008.

E : # edges

D : # dataflow facts

- A Tabulation Algorithm (**Time:** $O(|E||D|^3)$)

Precise Interprocedural Dataflow Analysis via Graph Reachability
Reps, Horwitz, Sagiv, POPL 1995



102

Demand-Driven CFL-Reachability

- We do not always need a transitive closure.

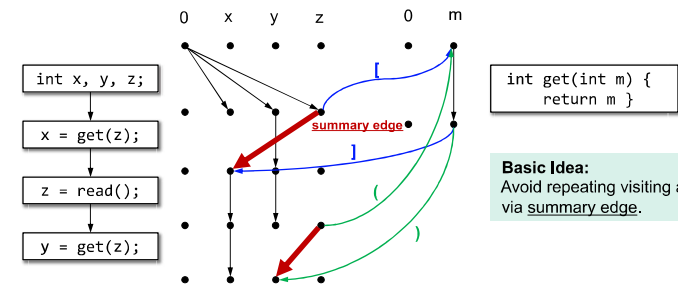
106

Which Method is Better?

- Computing a transitive closure
 - $O(1)$ query time;
 - $\sim O(n^3)$ preprocessing time --- **may not finish**;
- Search the graph for a path on demand
 - $\sim O(n)$ query time; --- **too long if we send queries frequently**;
 - $O(1)$ preprocessing;
- **Solution:** Indexing! An idea from the (graph) database community.

110

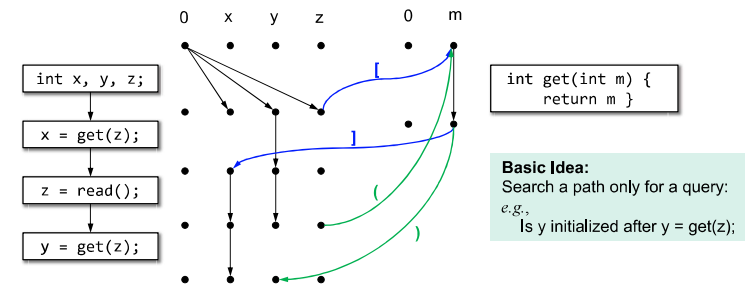
Tabulation Algorithm



PS: Edges between 0s are omitted

105

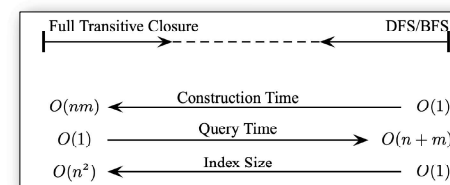
Demand-Driven CFL-Reachability



PS: Edges between 0s are omitted

107

Indexing Conventional Reachability



Work on DAG! Why?

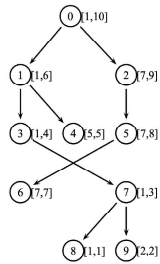
Two types of indexing!

- I. Compression of transitive closure
- II. Pruned Search

Store some intermediate results to speed up reachability query

113

Indexing Conventional Reachability



- **Label:** [low, high]
- high – post-order value
- low – minimum of the subtree
- $A \rightarrow B \Leftrightarrow \text{Label}_B \subseteq \text{Label}_A$

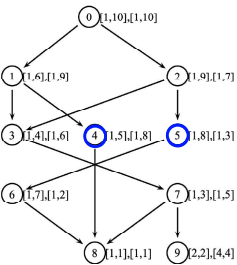
Generate $O(n)$ intermediate results in $O(n)$
to answer each query in $O(1)$

GRAIL: Scalable Reachability Index for Large Graphs

Hilmi Yildirim, Vineet Chaoji, and Mohammed J. Zaki, VLDB 2010

117

Indexing Conventional Reachability



- Generate multiple labels by randomly visiting the children of a node.
- Node 4: **[1, 5] [1, 8]**
- Node 5: **[1, 8] [1, 3]**
- **[1, 8] $\not\subseteq$ [1, 5] $\Rightarrow 4 \nrightarrow 5$**
- **[1, 8] $\not\subseteq$ [1, 3] $\Rightarrow 5 \nrightarrow 4$**

- **Q: How can the (un)reachability index help if there exists path between nodes?**

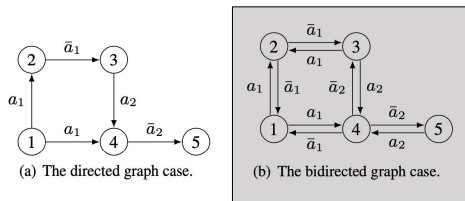
GRAIL: Scalable Reachability Index for Large Graphs

Hilmi Yildirim, Vineet Chaoji, and Mohammed J. Zaki, VLDB 2010

125

Beyond Context-Sensitivity

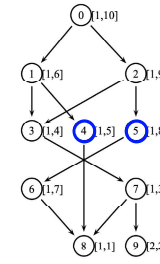
- CFL-reachability is often used for **flow-insensitive** alias analysis
- Alias analysis checks if two variables may have the same value



$$S \rightarrow SS \mid a_1 S \bar{a}_1 \mid \dots \mid a_k S \bar{a}_k \mid \varepsilon$$

129

Indexing Conventional Reachability



- **Label:** [low, high]
- high – post-order value
- low – minimum of the **subgraph**
- $A \rightarrow B \Rightarrow \text{Label}_B \subseteq \text{Label}_A$
- $\text{Label}_B \not\subseteq \text{Label}_A \Rightarrow A \nrightarrow B$

GRAIL: Scalable Reachability Index for Large Graphs

Hilmi Yildirim, Vineet Chaoji, and Mohammed J. Zaki, VLDB 2010

121

Indexing CFL Reachability



Indexing ... CFL Reachability for Context-Sensitive Program Analysis

Qingkai Shi, Yongchao Wang, Peisen Yao, Charles Zhang

In Proceedings of the ACM on Programming Languages (OOPSLA), 2022

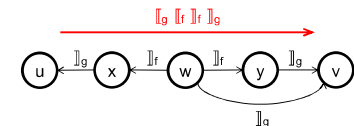
<https://doi.org/10.1145/3563339>

126

Beyond Context-Sensitivity

- CFL-reachability is often used for flow-insensitive alias analysis
- We do not consider the statement order when building graph

$$\begin{aligned} x &= w.f \\ y &= w.f \\ \textcircled{u} &= x.g \\ v &= y.g \\ w.g &= v \end{aligned}$$



133

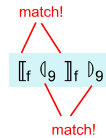
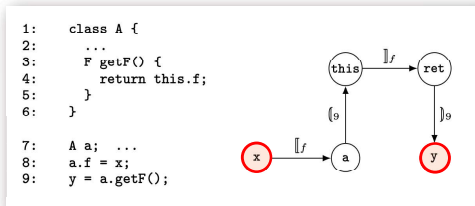
Context-Sensitive Alias Analysis?

- CFL-reachability can formulate context-sensitive analysis
- CFL-reachability can match field load and store in alias analysis
- How about a context-sensitive alias analysis?

136

Context-Sensitive Alias Analysis?

- Interleaved CFL Reachability! **Undecidable!**



They belong to two different CFLs!

Thomas Reps. Undecidability of context-sensitive data-dependence analysis. TOPLAS 2000.
Qirun Zhang and Zhendong Su. Context-Sensitive Data-Dependence Analysis via Linear Conjunctive Language Reachability. POPL 2017.

148

Computing Transitive Closure

- **Time:** $O(|V|^3/\log|V|)$; **Space:** $O(|V|^2)$



Subcubic algorithms for recursive state machines
Swarat Chaudhuri, POPL, 2008.

- Tabulation Algorithm for IFDS (**Time:** $O(|E||D|^3)$)

Precise Interprocedural Dataflow Analysis via Graph Reachability
Reps, Horwitz, Sagiv, POPL 1995

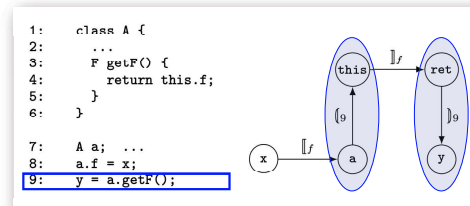


CFL-Reachability for program analysis may not be that hard,
compared to general cases!

151

Context-Sensitive Alias Analysis?

- Interleaved CFL Reachability!

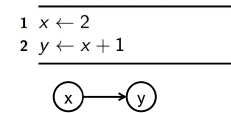


142

PART III: Fast CFL-Reachability for Context-Sensitive Data Dependence Analysis

Data Dependence Analysis

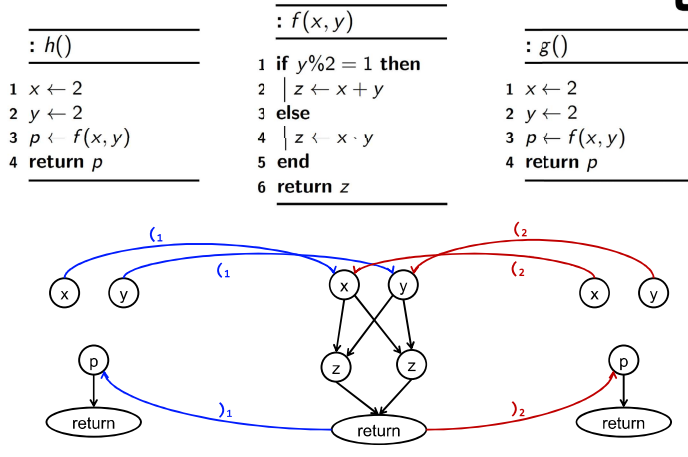
- Which variable depends on which others
 - Same-level dependence (vars from same func)
 - Diff-level dependence (vars from diff func)
- Identify def-use chains in a program
- LHS depends on RHS



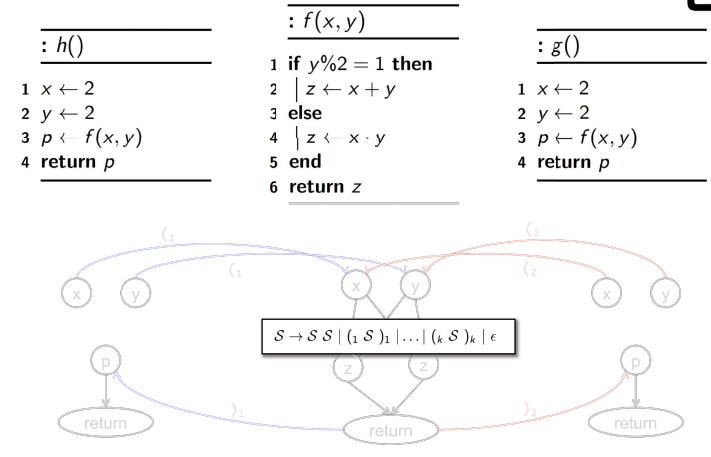
Your Turn!

```
1 result ← 0
2 for i ← 1 to n do
3   z ← x[i] · y[i]
4   result ← result + z
5 end
6 return result
```

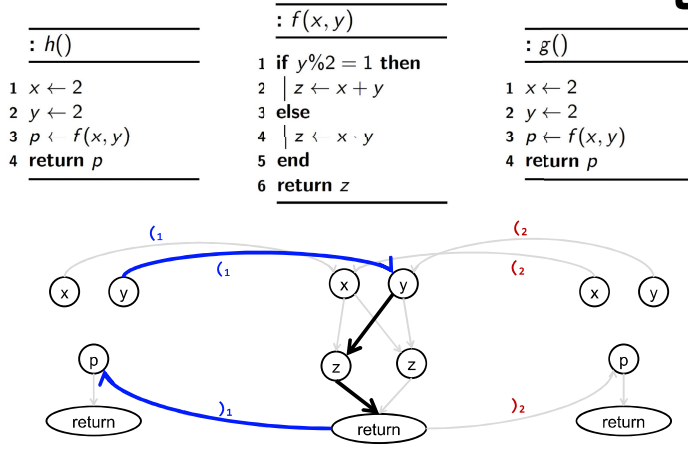
154



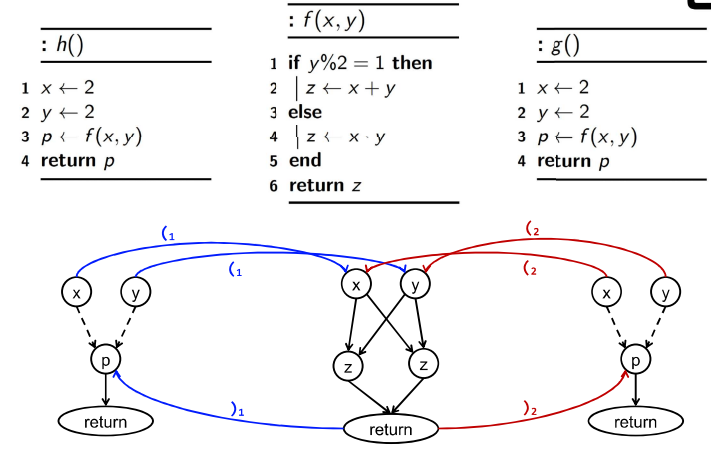
163



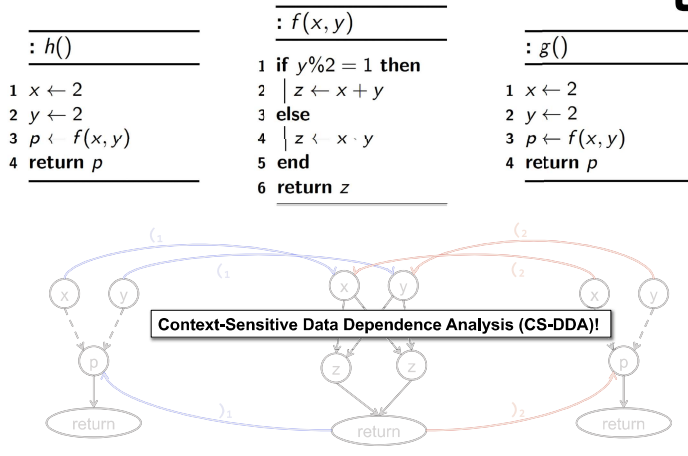
164



165



167

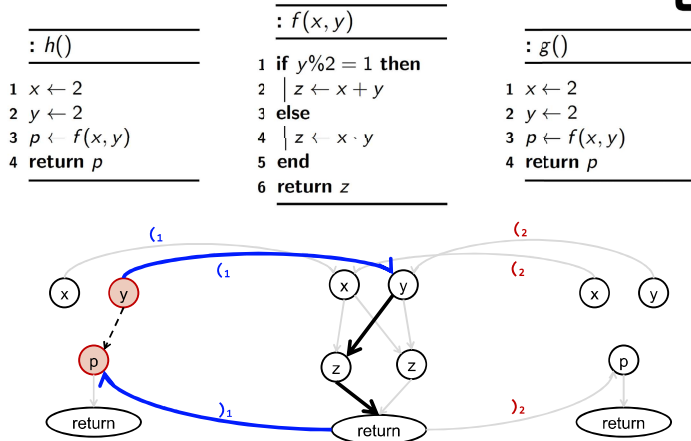


168

CS-DDA at the Same Level

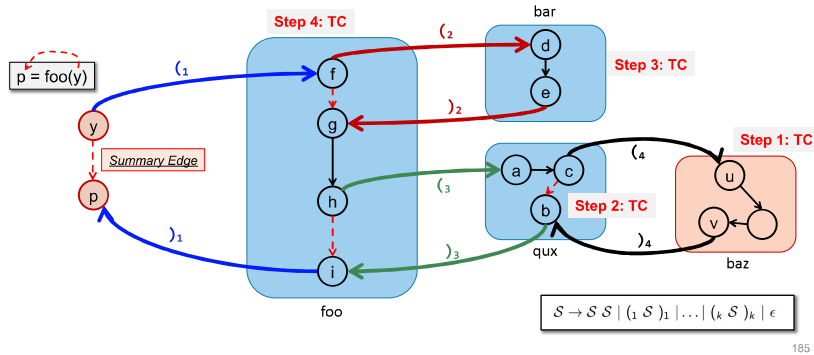
- **Goal:** Answer any same-level data dependence query in $O(1)$ after $\sim O(n)$ preprocessing

169



170

CS-DDA at the Same Level



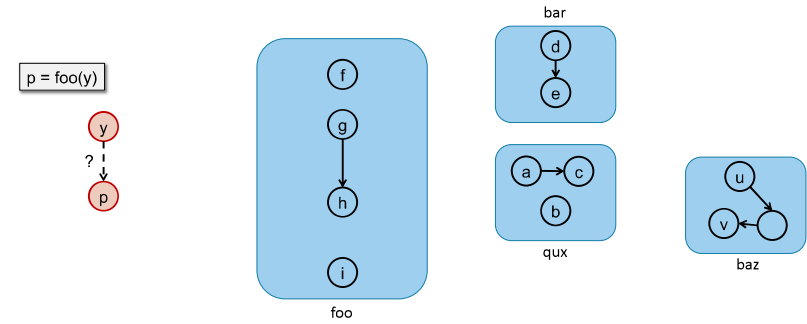
185

CS-DDA at the Same Level

- **Key Operation 1:** Computing TC for a function $\sim O(n)$
- **Key Operation 2:** Updating TC for a function $\sim O(\log n) \rightarrow \sim O(1)$
- **Key Observation:** The graph of each function is **sparse**, i.e., has a low treewidth (< 10)

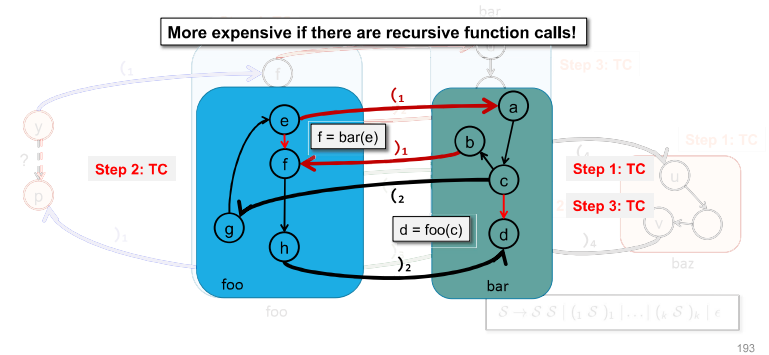
196

CS-DDA at the Same Level



171

CS-DDA at the Same Level

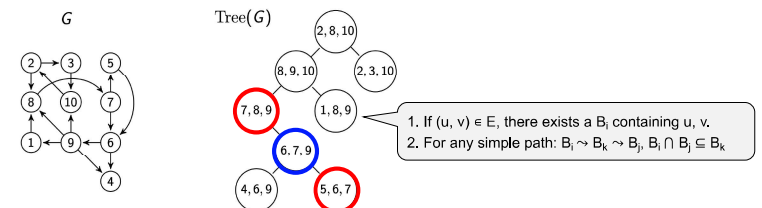


193

CS-DDA at the Same Level

Definition [Tree Decomposition]

Given a graph $G=(V, E)$, a tree decomposition of the graph $\text{Tree}(G)=(V_T, E_T)$ is a tree of bags $B_i \subseteq V$.

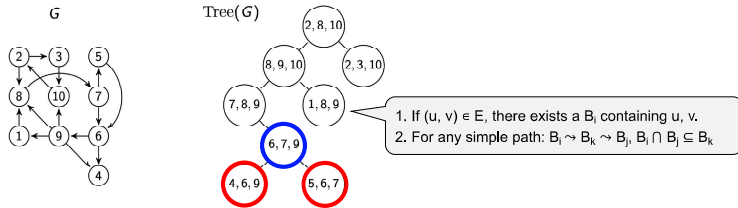


200

CS-DDA at the Same Level

Definition [Tree Decomposition]

Given a graph $G=(V, E)$, a tree decomposition of the graph $\text{Tree}(G)=(V_T, E_T)$ is a tree of bags $B_i \subseteq V$.

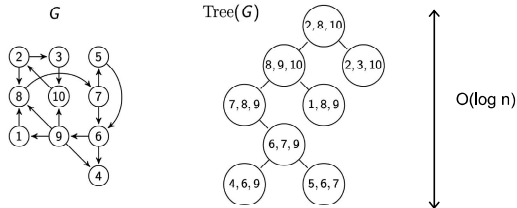


201

CS-DDA at the Same Level

Theorem [Tree Decomposition]

If the treewidth of a graph with n nodes is $O(1)$, $\text{Tree}(G)$ can be built in $O(n)$ time, and the height of $\text{Tree}(G)$ is $O(\log n)$.

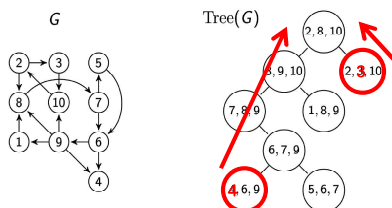


203

CS-DDA at the Same Level

Theorem [Efficient Reachability Query]

We can answer any reachability query in $O(\log n)$ time after pre-processing $\text{Tree}(G)$ using $O(n)$ time and space.

Computing TC for each bag is in $O(1)$ Computing TC for all bag is in $O(n)$ Computing TC for all bag in **bottom-up** orderQuery(x, y):

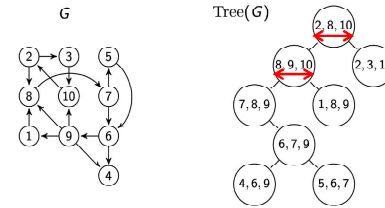
Climbing the tree to the top

212

CS-DDA at the Same Level

Definition [Treewidth]

The width of $\text{Tree}(G)=(V_T, E_T)$ is $\max|B_i| - 1$. The treewidth of a graph G is the minimum width of its tree decompositions.

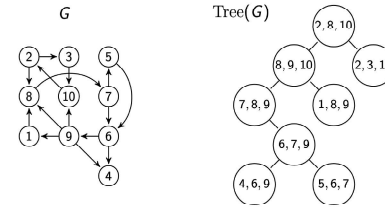


202

CS-DDA at the Same Level

Theorem [Efficient Reachability Query]

We can answer any reachability query in $O(\log n)$ time after pre-processing $\text{Tree}(G)$ using $O(n)$ time and space.

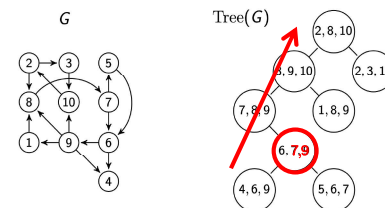
 $O(1)$ 

205

CS-DDA at the Same Level

Theorem [Efficient Update]

Assume a new edge (x, y) are inserted into G . We can update the reachability computed on $\text{Tree}(G)$ in $O(\log n)$ time if there is a bag $B \in \text{Tree}(G)$: $x, y \in B$.



Climbing the tree to update TC

214

CS-DDA at the Same Level

Theorem [Tree Decomposition]

If the treewidth of a graph with n nodes is $O(1)$, $\text{Tree}(G)$ can be built in $O(n)$ time, and the height of $\text{Tree}(G)$ is $O(\log n)$.

Theorem [Efficient Reachability Query]

We can answer any reachability query in $O(\log n) (\rightarrow O(1))$ time after pre-processing $\text{Tree}(G)$ using $O(n)$ time and space.

Theorem [Efficient Update]

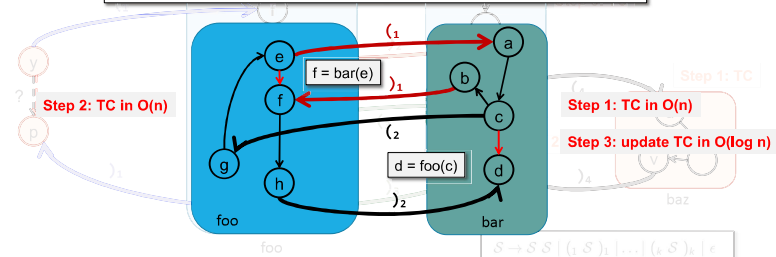
Assume a new edge (x, y) are inserted into G . We can update the reachability computed on $\text{Tree}(G)$ in $O(\log n)$ time if there is a bag $B \in \text{Tree}(G)$: $x, y \in B$.

K. Chatterjee, B. Choudhary, and A. Pavlogiannis. Optimal Dyck reachability for data-dependence and alias analysis. POPL 2018.

215

CS-DDA at the Same Level

TC is updated at most $O(a^2k) = O(k)$ times, where a is # params and returns at each call site, k call site in total. It takes $O(n + k \cdot \log n)$ time in total.



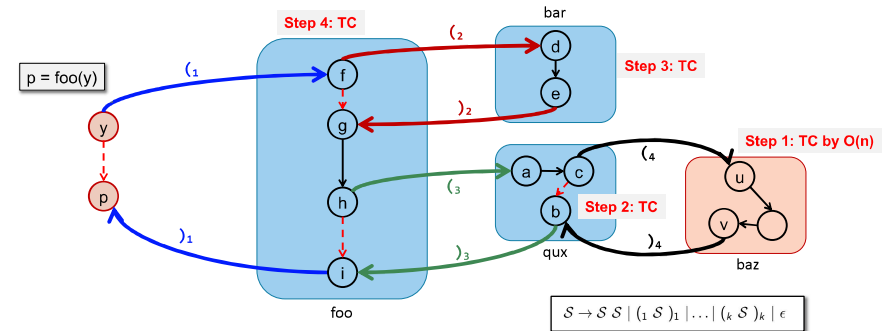
232

CS-DDA across Different Levels

- Answer any diff-level data dependence query in $\sim O(1)$ after $\sim O(n)$ preprocessing

235

CS-DDA at the Same Level

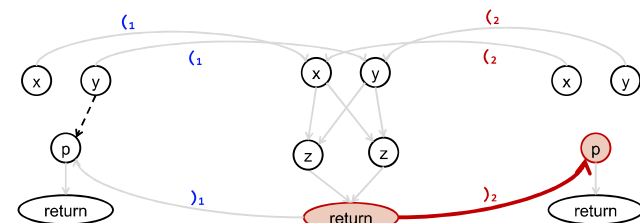
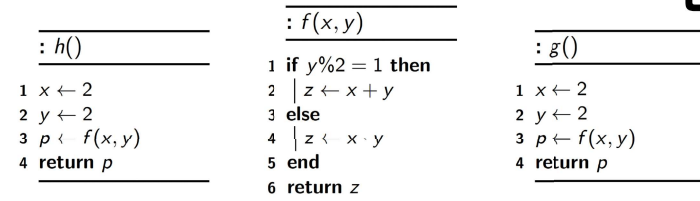


224

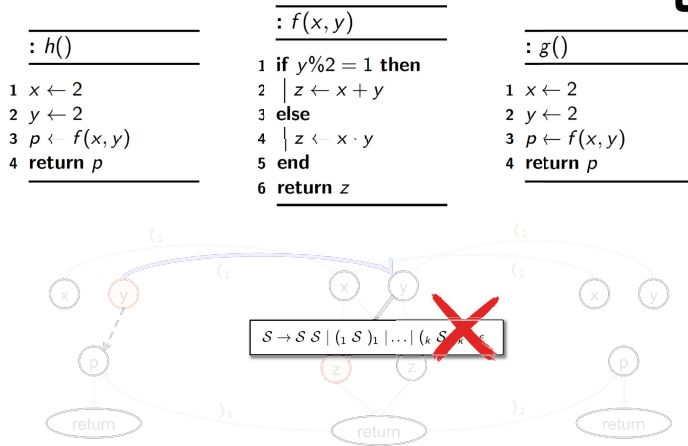
CS-DDA at the Same Level?

- Key Operation 1: Computing TC for a function
- Key Operation 2: Updating TC for a function
- Key Observation: The graph of each function is sparse with a low treewidth (< 10)
- Key Result: Answer any same-level data dependence query in $O(1)$ after a $O(n + k \cdot \log n)$ preprocessing

234



237

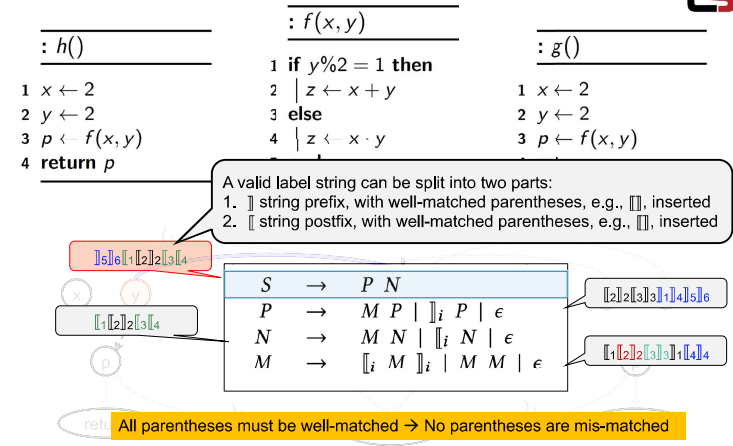


238

CS-DDA across Different Levels

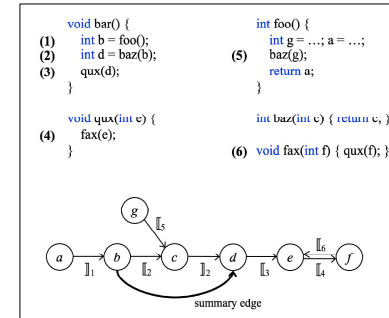
- Answer any diff-level data dependence query in $\sim O(1)$ after $\sim O(n)$ preprocessing
- Key Idea:**
 - (1) Use same-level CS-DDA to build summary edges
 - (2) Transform diff-level CS-DDA to conventional reachability
 - (3) Apply reachability indexing techniques

248



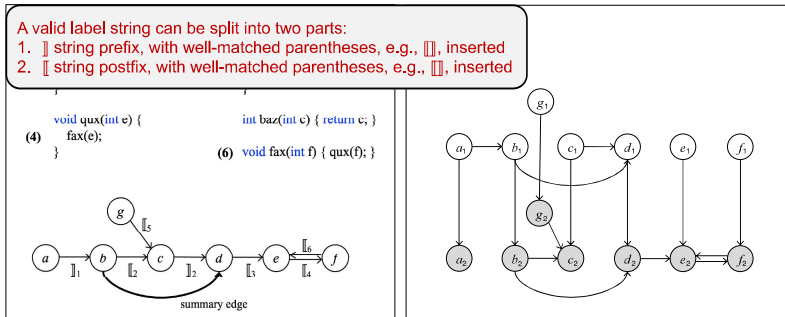
247

CS-DDA across Different Levels



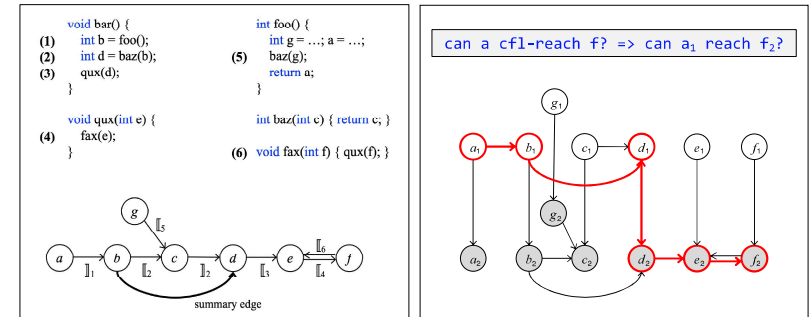
249

CS-DDA across Different Levels



253

CS-DDA across Different Levels



254